Attributing Probability to Language Models

Draft for AIDE 2026. Adrian Liu, Jul 24 2026, ~7k words

Language models are often said to implement conditional probability distributions over next tokens. A language model is said to implement a function that, for any string of words inputted, outputs a probability distribution indicating more and less likely next tokens given that string. The model's conditional probability distribution is learned from a corpus of existing data via an iterative process of determining which probability distribution best completes text that is in the given data.

The claim that language models implement conditional probability distributions in one sense demystifies these models. It is not surprising that a chatbot can complete "The capital of France is " with "Paris" if the underlying system outputs the most likely next word according to the model. The model can simply calculate, based on the corpus of text it has been trained on, that "Paris" is the word that most often follows the string "The capital of France is ". If we think that a plurality of relevant sentences in the training data are correct about the capital of France, then it is reasonable to think that a language model can output the correct answer.

But the claim that language models implement probabilities is much less demystifying when we consider the outputs of generative models given novel inputs. For instance, suppose I input all but the last paragraph of this paper into a language model before I write the final paragraph, so there is no ground truth about what the next words are. When the model generates a string of text based on this input, is it reasonable to say that the words generated reflect next-token probabilities? is it reasonable to say that, e.g. "In" is the most probable next word, if I haven't written "In", or any other word, as the next word of my paper? In general:

> LM-PROB: Is it reasonable to attribute probabilities to the outputs of language models?

LM-PROB is the topic of this paper. In this paper, I explore some strategies for answering LM-PROB in the affirmative, and argue that all of them face significant challenges.

LM-PROB is important to answer for at least two reasons. First, contemporary language models are implemented with complex, opaque structures (cf. [1]), exhibit emergent behaviors, and are in many ways difficult to interpret. Interpreting the output of a language model as a probability distribution explains a crucial component of the system in terms of a concept that we at least *somewhat* understand. Saying "a language model basically *predicts* how likely particular outputs are based on training data" leaves much unexplained, but it nonetheless provides a solid story for understanding the status of the outputs of the model. So it is imperative to check if we can make good on this story.

Second, LM-PROB bears on the following question of interest to interpretability research in artificial intelligence ([2, 3, 4, 5]):

> LM-EP: Is it reasonable to attribute epistemic states, like belief and credence, to language models?

If it is reasonable to attribute epistemic states to language models, the structures that instantiate these epistemic states will tend to have some probabilistic structures.

So obstacles to attributing probabilities to language models will in general also be obstacles to attributing belief or credence to these language models.

Here's the plan of the paper. In §1 I will introduce the formal basics of language models, demonstrating that we need not formally characterize them as outputting probabilities, but that they are by default taken to do so in the literature. This section should be read even by readers already familiar with language model formalism, since I lay out the particular assumptions I will be working with. In §2 I explain what I mean when I talk about attributing probability to a language model, and give two conditions that I take to be necessary for reasonably attributing probability to a language model. In §3 I consider some different strategies by which we might attribute probability to language models, and evaluate these strategies against the two conditions. In §4 I discuss the stakes of the discussion for answering LM-EP, and in §5 I discuss a way of interpreting language model outputs without attributing probability.

1 | Language Models

In this section I will give a basic formal characterization of language models. A language model is a function that takes two arguments, an input string x and a next-token candidate y, and outputs a real number. The input string x and next-token candidate y are both drawn from a vocabulary V of tokens: for simplicity we can think of the vocabulary as the vocabulary of a natural language like English. Each next-token candidate y is an element of V, and each input string x is a finite ordered list of elements of V. Thus in general the function is of the form

$$M(\cdot \mid \cdot) : V \times V^* \to \mathbb{R}, \tag{1}$$

where V^* is the set of all finite ordered lists of elements of V.

Contemporary language models are trained to be accurate at predicting the actual next tokens when given incomplete strings from some corpus $T \subseteq V^*$. For example, if "The capital of France is Paris" is a string in T, a model M that gives more accurate values of $M(\cdot \mid \text{The capital of France is })$ is preferred. Here, "more accurate" is measured relative to a function $\top$ that gives the true probabilities, based on the corpus, of what tokens follow the incomplete string. I'll suppose that $\top$ is of the form

$$\top(y \mid x) = \frac{\#_T(x + y)}{\#_T(x[\,])}, \tag{2}$$

where $\#_T(x + y)$ is the number of times the concatenated string $x + y$ occurs in the corpus, and $\#_T(x[\,])$ is the number of times a string of the form $x + \tau$, where τ is any single token, occurs in the corpus. This function will be defined only when $\#_T(x[\,]) > 0$.

Model training is a process in which a candidate model M is evaluated on each proper substring x of strings in the set T, and a loss function $\ell(M, x, y)$ calculates the distance of M from $\top$ for every pair (x, y). The model M is then modified iteratively so as to minimize its total loss over all substrings in the training data. Formally, training of the model seeks to minimize a quantity of the form

$$\sum_{x \in T} \sum_{y \in V} \ell(M, x, y). \tag{3}$$

Generally, the corpus T is split into a training set T_1 and a test set T_2. The model is trained on T_1, and data from T_2 "held out" from training so as to test the model's ability to predict correctly across the overall corpus T (e.g. to generalize beyond the data it was explicitly trained on).

Language models are often *normalized* so that their real-valued outputs are constrained to be in the unit interval $[0, 1]$, and so that the sum of their outputs on every candidate token, for any input string, is 1: $\sum_{y \in V} M(y \mid x) = 1$ for every $x \in V^*$. Together with the definition of the truth function $\top$, the normalization process ensures that an accurate model M will tend to give higher numbers on inputs y, x when $x + y$ occurs in T, and will tend to give lower numbers otherwise. Given that it is normalized and behaves in this way, the model M is often interpreted as giving, for each substring x in the training data, a probability distribution over next tokens y. That is, it is often said that $M(y \mid x)$ gives "the probability that y is the token that comes right after string x."

I will consider some examples to demonstrate just how standard this assumption that language models output next-token probabilities is in the literature. Savcisens and Eliassi-Rad ([6]) write:

> Consider an LLM $\mathscr{M}$ with a vocabulary V. The LLM $\mathscr{M}$ maps the input text x to a probability distribution over subsequent tokens, denoted $P_{\mathscr{M}}$:
>
> $$\mathscr{M}(x) = P_{\mathscr{M}}(\tau \mid x), \text{where } \tau \in V.$$
>
> For any token $\tau \in V$, the distribution $P_{\mathscr{M}}(\tau \mid x)$ denotes the probability that τ is the continuation of the sequence x.

Sometimes a language model is *defined* as a conditional probability distribution, like in Shanahan et al ([7]):

> the type of language model of interest here is a conditional probability distribution $P(w_{n+1} \mid w_1 \ldots w_n)$, where $w_1 \ldots w_n$ is a sequence of tokens (the context) and w_{n+1} is the predicted next token.

And Keeling and Street ([8]) write that the outputs of a large language model are

> probability distributions over tokens in the model's vocabulary such that the probability assigned to each token is an estimate of how likely each token is to succeed the input sequence.

For good measure, OpenAI's GPT 5.5 gives this definition for "large language model":[1]

> A large language model is a parameterized function $M_\theta : V^* \to \Delta(V)$ that maps a finite sequence of tokens from a vocabulary V to a probability distribution over the next token, thereby defining an autoregressive probability distribution over token sequences.

1. The prompt given: "Please give a simple mathematical definition of a large language model that would be standard in the computer science literature. The definition should be a single sentence. Do not output anything except for the sentence." The output was pasted directly, with no modifications (including TEX). Accessed July 21 2026.

Here are four details to keep in mind in further discussion. First, when I talk about a "language model", I will mean the formal structure described above. I will use the term "language model *architecture*" to describe any particular way in which the function $M(y \mid x)$ can actually be implemented. Modern language model architectures feed input data through many layers with different structures, based on transformer networks and perceptron networks. Thus, though we can abstractly discuss a language model as a black box and concern ourselves only with the input and output, it can also be of interest to probe into the architecture of language model implementations. In §3.5 I will discuss how my arguments extend to probing language model architectures.

Second, calls to contemporary language model interfaces like Claude and ChatGPT do not feed directly into a bare language model as described above, but through "orchestrators" that can route prompts to different, more purpose-built models as needed ([9]). The particular ways in which calls to these interfaces are structured is opaque for closed-source models. Nonetheless, the basic structure of each particular model that generates text in the end involves a language model architecture that implements an equation like the above.

Third, language models are today often fine-tuned on human feedback ([10, 11, 12]), with the effect of changing outputs so that characterizing the models as "minimizing loss over the training data" is not exactly right. This issue will occur in §3.2, but I will otherwise largely abstract away from it, because other problems for attributing probability to language models arise even if granting the idealization that models are trained to minimize loss over training data.

Finally, my target is *language* models, which underlie many of the most sophisticated AI systems nowadays. To the extent that processes said to be probabilistic are present in other AI systems (like diffusion models for image generation), similar concerns may apply, but I make no particular claims that the discussion generalizes.

2 | Attributing Probability

I've said that language models are often interpreted as giving the probability that a given string is followed by a given token. But what does it mean to interpret something as giving a probability?

As I will understand it, we interpret some function f as giving a probability (sometimes I will say we "attribute probability to f") if we interpret each output $f(x)$ as providing some measure of how probable or likely some proposition relating to x is, or providing some degree of confidence in the truth of the proposition. And we understand some function g as giving a *conditional* probability if we interpret each output $g(y \mid x)$ as providing some measure of how probable or likely some proposition relating to y is, supposing that some proposition relating to x is true. Thus we can say that a language model M is interpreted as a conditional probability distribution because we can interpret each output $M(y, x)$ as providing a measure of how probable or likely it is that y *is the next token*, supposing that x *is the string so far.*

Thus to interpret a model M as giving probabilities does not require, for instance, that the outputs are normalized, since non-normalized functions can play the same functional role. There is nothing special about the interval $[0, 1]$: multiplying every output of M by some real number, or adding some real number to it, or some other similar transformation, preserves the relevant structure needed to interpret

the output as indicating something about probability. Indeed, for computational reasons, instantiations of language models often work with log-probabilities, which are logarithms of our familiar $[0, 1]$ probabilities), in the range $(-\infty, 0]$. To make absolutely clear: to interpret a model as giving probabilities is not to require that the model outputs are probabilistically coherent (e.g. obeying the Kolmogorov axioms other similar axiomatizations ([13, 14])). It is easy to make the outputs of a language model probabilistic in a purely formal sense. What I care about is the interpretation of these numbers as having to do with judgments of events or propositions being more probable or less probable.

I am interested in this paper in the question of whether it is reasonable to interpret generative models as giving probabilities. I propose to operationalize this question with the following pair of necessary conditions:

1. **relevance**: we take the output $M(y \mid x)$ to be relevant to the conditional probability $\Pr(\mathsf{Q}_y \mid \mathsf{P}_x)$ of some propositions $\mathsf{P}_x, \mathsf{Q}_y$, functions of x and y.
2. **accuracy**: we evaluate the quality of the output $M(y \mid x)$ based on how *accurate* it is about how probable Q_y is given P_x.

The relevance criterion is motivated by the idea that probabilities must be understood as probabilities of *something* if they are to be something other than uninterpreted numbers. The accuracy criterion is motivated by the idea that part of what it is to understand some number as being "a probability of something" is to take it to give a better estimate of the target domain when it is accurate than when it is inaccurate.[2]

In all, I take relevance and accuracy to be rather weak conditions. If we did not take the output of a model to be relevant to any conditional probabilities of any propositions, then we would not take it to be a guide to any conditional probabilities. And if we did not think that the output $M(y \mid x)$ is better if it is more accurate and worse if it is less accurate, we would not take $M(y \mid x)$ as potentially a good guide to any truths about the world.[3] Neither of the conditions require attribution of any representation to the internal structure of a generative model. We need not think that propositions P_x and Q_y are represented. Nor do we need to think that some structure in the model is governed by a norm of accuracy. Nonetheless, I will argue that these conditions are difficult to meet.

2. I take this particular formulation from Joyce ([15]), but in one form or another it is a central commitment of the so-called "accuracy-first" or "accuracy-centered" approach in epistemology ([16, 17, 18]).
3. I am here committing to a kind of "constitutivism" about the concept of probability operative here, since I am saying that interpreting a number as a probability is partially constituted by taking a particular normative standard to apply. Hernandez Flowerman ([5]) has recently argued that similar constitutivist views about belief imply that large language models do not have belief. In brief, her argument is that constitutivist views about belief say, at least, that part of being a belief is to be held to a correctness standard (a belief is better if correct and worse if incorrect), but that there is no internal structure in language models to implement this correctness standard.

   I bring up Hernandez Flowerman's argument because it's helpful to see why it does not apply to my discussion. I am not discussing attributing *belief* to language models, only *probability*. And I am saying that to attribute probability to language models, it suffices for an observer to be able to interpret numbers extracted from a language model as probabilities. The numbers do not have to be governed by any particular epistemic norms within the system itself. So my criteria are less demanding than Hernandez Flowerman's.

3 | Attributing Probability to Language Models

Suppose I input this essay, without its final paragraph, into a language model M. Call this essay minus the final paragraph s. Suppose that of all possible next tokens, the one with the highest value given s is "In", with $M(\text{In} \mid s) = 0.6$. Can we interpret the output $M(\text{In} \mid s) = 0.6$. as giving a probability? In this section I will consider some ways in which we might interpret $M(\text{In} \mid s) = 0.6$. as giving a probability and judge them using the *relevance* and *accuracy* criteria. I will argue that all the methods face significant challenges.

3.1 | Indicative Conditional Probabilities on Training Data

One way to interpret the outputs of large language models is to say that they give *indicative conditional probabilities*. The indicative conditional probability of Q given P is the probability that Q is true, supposing that P is true. The *counterfactual* conditional probability of Q given P, which will be the focus of §3.4, is the probability that Q would be true supposing, perhaps contrary to fact, that P were to be true. To start, when an input $x[]$ is part of the training data, we can say:

> **TrainProb**: For a language model M, $M(y \mid x)$ is proportional to the probability, if a masked element of the form $x[]$ is drawn from T, that the last token is y.

Here a masked element $x[]$ is a string of tokens that comprises the substring x plus one last token at the end, which is hidden from view. By "drawing" a masked element $x[]$ from T, I mean drawing some element from T and then masking the last token, so that the result is $x[]$.

TrainProb, as a first-pass interpretation in terms of indicative conditional probability, works well when the strings $x[]$ are actually drawn from T. There are ground-truth probabilities for each masked element of T, based on how many strings are identical to $x[]$ except for the last token. But TrainProb cannot be right in general. For inputs not drawn from T, the premise, that x is a masked element of the corpus T, is always false. So the *relevance* criterion is not satisfied: $M(y \mid x)$ does not tell us anything interesting about strings drawn from outside the corpus T. And the *accuracy* criterion is not satisfied: different candidate values of $M(y \mid x)$ cannot be considered more or less accurate, since there is no ground truth to compare them against.

3.2 | Hypothetical Training Sets

In Reinforcement Learning with Human Feedback (RLHF), models are trained not only based on minimizing loss relative to a corpus, but also based on ratings of outputs given by human raters. A model can be trained to avoid politically-controversial statements, e.g., if human raters score such outputs lower, even if these statements have good loss scores on training data. Shanahan ([19]) suggests that language models fine-tuned via RLHF can be interpreted as modeling a hypothetical corpus including not only the actual corpus, but also hypothetical sentences that *would* be written by human raters, where the fine-tuned model learns what these sentences are based on the human ratings:

> Another way to think of an LLM that has been fine-tuned on human preferences is to see it as equivalent to a raw model that has been trained on an augmented dataset, one that has been supplemented with a corpus of texts written by raters and / or users.

If fine-tuning justifies seeing a language model as having been trained on a hypothetical augmented dataset T^+, then we could implement **TrainProb** with T^+ in place of T, and pretend that we have a corpus T from which masked elements can be drawn.

> **TrainProb**$^+$: For a language model M, $M(y \mid x)$ is proportional to the probability, if a masked element of the form $x[]$ is drawn from T^+, that the last token is y.[4]

Interpretationally, we run into two problems here.[5] First, how should we interpret this augmented dataset? It has some strings that are in the actual underlying dataset. And then it has some "hypothetical strings" that are not actually in an underlying dataset but are ones that raters *would* write. So the probability that a masked string from this combined data-generating process ends in a particular token is perhaps best interpreted as the probability that a string that *either* exists in the training data *or* would hypothetically be generated has a particular last token. I don't think this is an insurmountable interpretational challenge, but it seems awkwardly disjunctive to me, and in particular makes the *accuracy* criterion hard to evaluate. For example, suppose a particular racist sentence $x + y$ actually occurs in training data but is dispreferred by raters, who downvote it due to its objectionable content. If the resultant fine-tuned language model gives $M(y \mid x)$ a low score, do we say it is accurate or inaccurate? On the one hand, the model correctly registered that the string is dispreferred by raters. On the other hand, the string *does* appear in the training data. So the relationship between the training data and the fine-tuning must be more subtle, and it's not clear to me that there is an interpretation that results in a well-defined underlying set that we can say M models.

The second problem is a reprise of the problem that faced TrainProb: what happens when we outrun the training data? In Shanahan's example, the augmented training dataset is larger than the actual dataset, but still finite (since a finite amount of fine-tuning is performed). So input strings can still outrun the entire augmented corpus, at which point there is once again no ground-truth answer as to what token comes next. Here it is no help to say that the model gives the probability that a token comes next if the string is drawn from the augmented dataset, if we're considering strings that, by hypothesis, do not occur in even the augmented dataset.

4. In an earlier draft I presented this in counterfactual terms, as a version of a counterfactual conditional interpretation. But even though the language model is not actually trained on the set T^+, we can still say that T^+ exists as a mathematical object, so it is coherent to "actually" draw samples from it. Furthermore, the problems that face TrainProb$^+$ bear much more similarity to the problems facing indicative conditional accounts than counterfactual conditional accounts.
5. There is also the problem that once a model is fine-tuned on human feedback, its overall training is no longer calibrated to the training data. My discussion will abstract away from this issue, since we're considering that we might be able to evaluate the model based on a hypothetical augmented dataset, on which the model could (hypothetically) be calibrated.

3.3 | Indicative Conditional Probabilities on Existing Strings

What if we appealed to the fact that strings drawn from outside training data are similar in relevant ways to strings drawn from the training data? After all, it is this sort of idea that underlies classifiers in machine learning. For example, an image classifier that can classify images as of cats or dogs can do so even for images of cats and dogs that are outside the training data.

Formally, a classifier is a function that takes two arguments, an input x and a classification c, and outputs a real number. The input x is drawn from some set X and the classification is drawn from some set C of classifications. For instance, if X is a set of images of cats and dogs, and $C = \{cat, dog\}$, then we can understand a classifier $M(c \mid x)$ as giving indicative conditional probabilities on training data:

> **Classifier Interpretation on Training Data**: For a classifier model CM, $CM(cat \mid x)$ is proportional to the probability, if x is an image drawn from the training data, that the image is of a cat.

A well-trained classifier model will, if we give it images of cats or dogs that are sufficiently similar to the ones in the training data, give high values of $M(cat \mid x)$ to images of cats and high values of $M(dog \mid x)$ to images of dogs. So both the *relevance* and *accuracy* criteria can be satisfied, and we can interpret the classifier as a probability even outside the training data:

> **Classifier Interpretation Outside of Training Data**: For a classifier model CM, $CM(cat \mid x)$ is proportional to the probability, if x is an image of a cat or a dog that is similar to images in the training data, that the image is of a cat.

Formally, a language model can be thought of as a classifier where the set C of classifications is the vocabulary V itself. The "category" that any given string is classified as is simply the most likely next token for the string. Given this, can we analogously understand the outputs of a language model to be probabilistic as follows?

> **IsProb**: For a language model M, $M(y \mid x)$ is proportional to the probability, if $x[\,]$ is a masked string drawn from a target domain that is similar to the training data, that the last token is y.

IsProb does not work for language models. In the classifier case, all images, even those not in the training data, are of cats and dogs, and thus for every image, one of the labels is correct. Thus in the classifier case, the accuracy criterion is satisfied. If we were to input an image of a frog into the classifier, we would not expect accurate estimations from the classifier. Or at least, if the classifier accurately classified the image as unlikely to be of a cat, it would be at the expense of inaccurately classifying it as likely to be of a dog. Thus, outside of the class of images that are correctly labeled as "cat" or "dog", the interpretation of the classifier as a probability is not readily applied.

In the case of a language model, the analogue of the class of images correctly labeled as "cat" or "dog" is the class of strings where there is a correct answer about what token comes next. For any input string that is not a substring of a larger string that continues past it in some existing corpus (call this a *left-substring*), there will be no correct answer about what token comes next. So any novel input string to the

language model (by novel I simply mean that nobody has written the string before) will be outside the class of strings where there is a correct answer about what token comes next. If there is no correct answer about what token comes next, then the *accuracy* criterion cannot be satisfied, since the model output cannot be evaluated on the basis of being more or less accurate about what token comes next. So for novel strings of text, the IsProb interpretation of language model outputs does not succeed. Appealing to a target domain beyond the training data does substantially increase the domain over which the probability interpretation applies, extending it to all strings that are left-substrings of strings that have been written. But it still cannot account for what a language model is doing when given a string nobody has written before.

3.4 | Counterfactual Conditional Probabilities

Suppose that instead of understanding the output of a generative model M as indicating the probability that a given string x *is* followed by a token y, we understand it as indicating the probability that x *would be* followed by y if x were to be generated by some data-generating process.

> **WouldProb**: For a language model M, $M(y \mid x)$ is proportional to the probability, if data generating process $\mathscr{D}$ were to output a masked string $x[]$, that the last token of x would be y.

The main interpretational difference between IsProb and WouldProb is that in IsProb we suppose that the masked strings actually exist in the target domain already, and we take M to estimate particular features of these masked strings. On the other hand, in WouldProb we suppose only that these strings *could* be generated by some data-generating process, and we take M to estimate the properties that generated strings *would* have. Thus WouldProb is not immediately susceptible to the objection that proved difficult for IsProb: that any novel string is in no "existing corpus."

But understanding a language model M using WouldProb requires specifying the data generating process $\mathscr{D}$ that we take M to model. For purpose-built language models trained to be good at particular domains with specifiable correct answers, we could take the data-generating process to be one that generates all and only pairs of questions and their correct answers. For instance, the pattern $2 + 1 = 3, 2 + 2 = 4, 3 + 2 = 5, \ldots$ might imply a data generating process that outputs true instances of the schema $x + y = z$, where x, y, z are natural numbers.[6] As long as only questions of the right form are input into the model, this interpretation might suffice.

The issue with identifying domain-specific data generating processes is that it does not give a general account of what data generating process is being modeled, for *any* input string. For input strings that don't fit the schema of the domain-specific data generating processes, WouldProb does not satisfy the *relevance* criterion, since the string in question is *not* generated by the domain specific process. So if the model under consideration is a more general-purpose one that takes in arbitrary strings, then although we could only say of some particular inputs x that $M(y \mid x)$ should be

6. When the pattern is input as part of the prompt, as in the case of in-context learning ([20, 21]), there is an additional question of whether to locate the data generating distribution in the few-shot prompt or in the base model. I will abstract away from this difficulty for the purposes of the paper.

interpreted as modeling a specific data-generating process, this interpretation would not extend to the input space as a whole.[7]

One way to give WouldProb broader applicability is to interpret the entire training data itself as produced by a single underlying data-generating process, and to understand the model as modeling not just the training data, but instead *whatever* data generating process was responsible for generating the training data. According to this understanding, an accurate model not only fits the training data well, but is good at predicting the distribution of outputs from the underlying data generating process.

Any such strategy faces metaphysical and epistemic hurdles in characterizing the underlying data generating process in question. At the metaphysical level, there are many ways to truly characterize a process that generated the training data. Indeed, for a corpus of textual training data, the following all seem to have valid claims to be a process that generated the data:

1. The process of humans generating all and only the content in the corpus.
2. The process of humans generating written content between 1500 and 2026 CE.
3. The process of humans generating content.

There is no obviously principled way to privilege one as *the* process that generated the data. But a choice must be made, on some grounds, in order to identify what particular data-generating process we interpret a language model to be modeling. This metaphysical problem can be seen as an instance of the well-known reference class problem that faces interpretations of probability in general ([23]) and of probability in machine learning in particular ([24]). as Hu ([24]) writes:

> We face a reference class problem whenever we want to assign a probability to a particular individual case, but the case at hand may be classified in many different ways, each of which suggests for it a different probability. So, we must ask: which of these different conditional probabilities referring to different reference classes gives the right unconditional probability?

For our purposes, the reference class problem can be read as a problem about what the "correct" distribution is, or as a problem about what the "best for our purposes" distribution is. One framing is more metaphysical, while the other is more normative, but both are significant challenges.

At the epistemic level, for any choice of underlying data-generating process, there will be the question of whether we have good reason to think that the model successfully models *that* process. At one extreme, if the process is simply whatever processes generated all and only the content in the corpus, then any model that fits the data fits the process, but our interpretive situation on any strings outside the training data is exactly that of TrainProb. At the other extreme, if the data generating process is "humans generating content" or something similar, we would need some reason

7. Another point to note is that the way the loss functions are built may make it more natural to interpret the output of a model as giving the probability that "z" is the next token given that "x + y" is the input; rather than the probability that z is the correct answer if "x + y" is the question ([8, 22, 19]). Still, if a language model becomes accurate enough on the latter question, then interpreting the model in the latter way still conforms to *relevance* and *accuracy*, and we may give a story like: accurately estimating the latter probabilities is a good means toward the end of estimating the former probabilities.

to think that the models could adequately model this process in general, even into the future. And this would involve strong inductive claims about the resemblance of future human-generated content to past human-generated content.[8]

An attractive answer may be found at a middle ground. For instance, we might posit a data-generating process describable as $\mathscr{D}$ = "humans generating written content between 1500 and 2026 CE" in a way that allows us to think that we could imagine sampling from this data generating process to get strings that *could have* been generated by humans 1500 and 2026, even if they in fact were not. And then we could say that a model output $M(y \mid x)$ estimates the conditional probability, if $x[]$ is a string that might have been generated by humans between 1500 and 2026 according to $\mathscr{D}$, that the next token is y. This would satisfy the *relevance* criterion. To satisfy the *accuracy* criterion, we could say that the accuracy of the model is calculated against the actual probability that a string of the form $x[]$ sampled from $\mathscr{D}$ is $x + y$.

Though such a middle ground is possible in principle, in practice the epistemic and metaphysical problems seem serious. On the epistemic side, the problem seems especially bad for operationalizing the accuracy criterion: how do we determine how accurate a model is at predicting a probability from a distribution that we can't actually sample from? On the metaphysical and normative side, the reference class problem becomes our problem again: what makes the particular characterization of the distribution $\mathscr{D}$ that we've chosen of particular significance, so that it *matters* that the model is sampling from this distribution and not some other?

3.5 | Attributing Probability to Internal-Layer Structures

So far I have been discussing the outputs of language models, described as the abstract structure $M(\cdot \mid \cdot)$. Contemporary *large* language models implement the language model function M through a complex internal architecture involving many layers of linear and nonlinear transformations of data. Recent work in interpretability research has used probes to attempt to extract semantically-significant data from internal architectures implementing language models ([27, 28, 29, 2, 3, 6, 30]). These probes are often remarkably able to find patterns in internal layers that appear to track truth, and their successes raise the prospect that we could attribute probabilities to internal layers of language model architectures.

Any proposal for attributing probabilities to internal layers of language model architectures will face the challenges that this paper has so far posed for attributing probabilities to last-layer outputs. Indeed, suppose that a probe is able to extract values in internal layers of a language model that correspond to probabilities of some events over a corpus. When we subsequently input novel strings into the language model, what probabilities would the probe-extracted values represent? The chain of possible interpretations, and worries about those interpretations, explored in §3 arise here just as they did for last-layer outputs.[9]

8. Especially given that predictions can be *performative*, in that they can change the future ground truth from what it otherwise would have been ([25, 26]). In this context, it's plausible that the overall shape of human-generated textual content going forward changes when language model outputs are in the picture, compared to when they are not.
9. For a restricted class of probes, we may be able to unproblematically attribute probabilities to extracted

4 | Attributing Credence to Language Models

In the introduction I distinguished between LM-PROB, the question of whether we can reasonably attribute probability to language models, and LM-EP, the question of whether we can reasonably attribute epistemic states to language models. I also suggested that LM-PROB is in some ways easier to answer in the affirmative, since in order to attribute probability to a model output we need not think that the outputs are used in distinctively epistemic ways in the functioning of the model. The topic of this paper has been not epistemic states *per se*, but about probability representations. However, to the extent that particular probabilistic structures within a language model architecture can be said to be involved in epistemic states of the model architecture, the worries I've raised will apply to attempts to answer LM-EP in the affirmative.

To help see why, I will here present criteria for attributing representation to AI models from Harding ([28]). Harding's focus is representation in general: since beliefs and credences are representational, the criteria apply. Harding proposes the following three principles, which she attributes to the philosophy of cognitive science, for judging whether AI systems represent properties. In her formalism, D is a "task", represented as a set of inputs (like input strings to an language model), and $h(D) = \{h(s) : s \in D\}$ is the pattern of activations that the system has for inputs in D. She writes:

> I suggest that a pattern of activations $h(D)$ represents a property Z if the following three criteria hold:
>
> **The Information Criterion**: The pattern of activations $h(D)$ bears information about Z.
>
> **The Use Criterion**: The information the pattern of activations $h(D)$ bears about Z is used by system S to perform task D.
>
> **The Misrepresentation Criterion**: It should be possible (at least in principle) for the activation vector $h(s)$ to misrepresent Z with respect to an input $s \in D$.

Harding's information criterion corresponds to my relevance criterion, since my relevance criterion requires that the output $M(y \mid x)$ bear information about conditional probabilities for *some* events. Her misrepresentation criterion bears similarities to my accuracy criterion, since a less accurate model in a sense misrepresents the true probabilities of the target domain.[10]

However, her use criterion importantly has no analogue in my conditions. This is because I do not need the system, i.e. whatever computational structure the language

values. For instance, consider a probe trained to detect if a model makes a distinction between true and false propositions ([2, 3, 6, 30]). Given a setting that guarantees that every input to the model being probed is a proposition that is either true or false, then we may be able to reasonably interpret probed values as probabilities that the proposition input is true. In effect, the guarantee that every input is a true or false propositions reduces these sorts of cases to classifier cases, and we've already seen above that probability attributes can be unproblematic for classifiers.

10. However, my accuracy criterion also requires that we evaluate the quality of the output of a model based on its accuracy, not just that it be possible for the model to be inaccurate. Thus my accuracy criterion also differs somewhat from the accuracy criterion in [4], since they require only that models are *in fact* accurate, not that accuracy is taken as a normative standard.

model is embedded within ([1]), to use information about conditional probabilities to do any particular task. I need only that an observer can extract numbers from the model or the model architecture and reasonably interpret these numbers as probabilities.

The analogues for *accuracy* and *relevance* are important, as is the lack of an analogue in my framework for Harding's *use*. The reason why the analogues for *accuracy* and *relevance* are important is straightforward. Suppose that *relevance* and *accuracy* are necessary conditions for attributing credence to language models. Then the failure of particular structures to satisfy relevance and accuracy may disqualify them from underpinning an epistemic state. For instance, suppose we should not attribute a conditional credence $cr(\mathsf{Q}_y \mid \mathsf{P}_x)$ to a part of a language model architecture if nothing in that part of the architecture satisfies *relevance* and *accuracy*. Then if we cannot find a way to interpret a part of the architecture as a conditional probability $\Pr(\mathsf{Q}_y \mid \mathsf{P}_x)$ because every candidate fails to satisfy *relevance* and *accuracy*, then we will also fail to find a part of the architecture that meets the conditions for being a credence.

The reason that the lack of analogue for a *use* criterion is relevant is that it may be worrisome to find situations in which the *use* criterion is satisfied by a structure, but *relevance* (information) and *accuracy* (misrepresentation) are not. For instance, we may find that some structure within a language model architecture is playing a belief- or credence-like role in the functioning of the system (thus satisfying Harding's *use* criterion), even though it involves representations that cannot be interpreted as probabilities in general. In these circumstances, we may want to say that the language model architecture has a pathological version of beliefs or credence: pathological because they play a functional role of belief or credence even though they do not have the right representational structures to play these functional roles in the right way.

In general, it seems plausible to me that being able to attribute probability is a necessary condition for being able to attribute belief or credence to a language model or language model architecture. But there may be ways to get around this connection. Language model probabilities are often attributed over different sets than credences and beliefs are: for instance, language model probabilities are typically understood to be next-token probabilities, while credences and beliefs attributed to models have propositions or events as their content ([22, 8]). Keeling and Street ([8]) consider a number of candidates for bridge principles between the next-token probabilities attributed to a language model and credences attributed to the model. Although their candidates all depend on interpreting language model outputs as next-token probabilities, there could in principle be plausible bridge principles that do not require such an interpretation of the next-token probabilities. Of course, we would still have to ensure that the resultant understanding of the credence attribution does not fall prey to the issues for probability interpretation raised in this paper.

5 | Simulation Interpretations

I've argued in this paper that attempts to understand language model outputs as probabilities face several interpretive challenges. Different ways of saying that an output $M(y \mid x)$ corresponds to a probability all have difficulty with the *relevance* and *accuracy* criteria. To end, I will consider a strategy that explicitly rejects the demand of giving an interpretation of a language model's outputs in terms of giving the probabilities of some events.

Shanahan et al ([7], see also [31, 32]) suggest that language models, together with dialogue prompts that suggest a personality, style, goal, or other parameters, for generated text to imitate, can simulate different "characters". Metaphorically, different textual continuations can implement different possible characters for the textual output to "role-play", and the model's distribution over next tokens can be seen as a superposition of different characters, or "simulacra", that the textual output can instantiate. According to a *simulation* interpretation, the function M should not be interpreted as a probability at all. Instead it should be seen as a function that provides candidate continuations for input strings. The numbers attached to these candidate continuations can be interpreted as a measure of similarity: for instance, if y has a higher score than y', perhaps $x + y$ is a string that is more similar in some way to strings in the training data than $x + y'$. But this similarity does not suggest claims about how probable any events are, and thus the numbers do not represent probabilities.

It is consistent with the simulation interpretation that the real numbers that the model outputs, before conversion to actually-generated text, are properly interpreted as probabilities. Even if the final product of generated text is interpreted as a possible continuation or simulation, the numerical value of $M(y \mid x)$ can still be a probability. Indeed, Shanahan et al interpret the mathematical object $M(y \mid x)$ as outputting a probability, even as they argue that the model is used for simulation. However, for present purposes I think one of the advantages of the simulation account is that it gives an alternative to interpreting model outputs as probabilities, and thus avoids the problems for the interpretations I've so far discussed. Instead of interpreting model outputs as probabilities, we can interpret model outputs as proposing continuations of strings, and proposing different continuations with different strengths.

> **SimProp**: For a language model M, $M(y \mid x)$ is proportional to the relative strength of the proposal that y be the token following string x.

We can then understand the training of the model as favoring string continuations that lead the entire string to resemble the training data in particular ways (the loss function can be seen as measuring similarity). What this resemblance or similarity amounts to is not a trivial matter, and would be a considerable task for interpretability research. However, this similarity-based interpretation of model outputs sidesteps the problems that arose when taking model outputs to be probabilities.

In particular, note that neither the relevance criterion nor the accuracy criterion are as pertinent to a simulation-based interpretation. If a model output is a distribution over proposals for some way to continue a string, it need not represent any particular probability. There are no events in the world that we take the numbers to estimate and thus provide information about. Similarly, the accuracy criterion is not pertinent, because a distribution over proposals to continue a string is not the type of thing that it makes sense to say is "more accurate" or "less accurate."

We might worry about how we can even understand the training of language models if we are dispensing with the idea that model outputs give a probability distribution over likely next tokens, and that they are trained to be maximally accurate over a training set. I think this worry poses a substantive challenge to the simulation interpretation, and will consider two ways in which it could be addressed. First, we can say that language models *are* trained to outputs probabilities over next tokens when inputs are drawn from training data, but that we should not interpret outputs as

probabilities when inputs are not drawn from training data. In other words, we could say that language models should be interpreted piece-wise. When x is in the training data, $M(y \mid x)$ is a probability according to indicative probability interpretations, for example; and when x is not in the training data, $M(y \mid x)$ gives a measure of how "strongly" M proposes y as the continuation of string x:

$$M(y \mid x) \propto \begin{cases} \Pr(y \text{ is next token} \mid x[] \text{ is masked string}) & x \in T \\ \text{Strength of proposed continuation } y \text{ to } x & x \notin T \end{cases} \tag{4}$$

Such an approach is worryingly disjunctive. On what grounds can we say that a single language model does two different things based on the provenance of the input? It would be one thing if M implemented a disjunctive function that did different things on numbers than on text, for example; but there is no intrinsic difference between string in a training data and outside of it, and nothing in the formal structure of a language model respects the distinction. So the distinction fails to be well motivated.

An alternative response to the worry is to say that language models do not output probabilities even on training data, and are instead *always* simply proposing possible continuations with different strengths. On this second response, we would interpret the loss function as enforcing some kind of similarity between generated strings and strings in the training data, but not to maximize any sense of predictive accuracy for model outputs compared to the training data. This is *prima facie* a coherent position, and it may well be the best way out of the problems that face probability interpretations. But we should immediately note that it amounts to answering LM-PROB in the negative, saying that it is not reasonable to attribute probabilities to the outputs of language models. This negative answer is in conflict with the default way in which much of the literature interprets language models at a theoretical level. It also forecloses the possibility of using a probability-based understanding of language model outputs to understand what these models are doing when they generate distributions over next tokens. So this response has its explanatory costs.

6 | Conclusion

language models are standardly interpreted as outputting conditional probability distributions over next tokens. Such an interpretation plays a central role in interpreting what language models do in general. But if language models can be interpreted as outputting probabilities, then the outputs must be relevant to some probabilities, and evaluable based on how accurate they are about some domain. And it seems to me that there is just no straightforward answer for what these probabilities and what the domain might be.

References

[1] Kathleen A. Creel. "Transparency in Complex Computational Systems". en. In: *Philosophy of Science* 87.4 (Oct. 2020), pp. 568–589. DOI: 10.1086/709729.

[2] Amos Azaria and Tom Mitchell. "The Internal State of an LLM Knows When It's Lying". In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 967–976. DOI: 10.18653/v1/2023.findings-emnlp.68.

[3] B. A. Levinstein and Daniel A. Herrmann. "Still No Lie Detector for Language Models: Probing Empirical and Conceptual Roadblocks". en. In: *Philosophical Studies* 182.7 (July 2025). arXiv:2307.00175 [cs.CL], pp. 1539–1565. DOI: 10.1007/s11098-023-02094-3.

[4] Daniel A. Herrmann and Benjamin A. Levinstein. "Standards for Belief Representations in LLMs". en. In: *Minds and Machines* 35.1 (Dec. 2024), p. 5. DOI: 10.1007/s11023-024-09709-6.

[5] Camila Hernandez Flowerman. "AI, LLMs, and the normativity of belief". en. In: *Synthese* 207.4 (Mar. 2026), p. 132. DOI: 10.1007/s11229-026-05475-3.

[6] Germans Savcisens and Tina Eliassi-Rad. *The Trilemma of Truth in Large Language Models*. June 2026. DOI: 10.48550/arXiv.2506.23921.

[7] Murray Shanahan, Kyle McDonell, and Laria Reynolds. "Role Play with Large Language Models". In: *Nature* 623.7987 (Nov. 2023), pp. 493–498. DOI: 10.1038/s41586-023-06647-8.

[8] Geoff Keeling and Winnie Street. "On the Attribution of Confidence to Large Language Models". In: *Inquiry* 69.6 (July 2026), pp. 2918–2944. DOI: 10.1080/0020174X.2025.2450598.

[9] David Dohan et al. *Language Model Cascades*. July 2022. DOI: 10.48550/arXiv.2207.10342.

[10] Amelia Glaese et al. *Improving Alignment of Dialogue Agents via Targeted Human Judgements*. Sept. 2022. DOI: 10.48550/arXiv.2209.14375.

[11] Long Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback*. Mar. 2022. DOI: 10.48550/arXiv.2203.02155.

[12] Timo Kaufmann et al. *A Survey of Reinforcement Learning from Human Feedback*. arXiv:2312.14925 [cs.LG]. Dec. 2025. DOI: 10.48550/arXiv.2312.14925.

[13] Hájek, Alan. ""Interpretations of Probability"". In: *Stanford Encyclopedia of Philosophy, Winter 2022* ().

[14] James M. Joyce. "A Nonpragmatic Vindication of Probabilism". en. In: *Philosophy of Science* 65.4 (Dec. 1998), pp. 575–603. DOI: 10.1086/392661.

[15] James M Joyce. "Why Evidentialists Need not Worry About the Accuracy Argument for Probabilism". en. 2013.

[16] James M. Joyce. "Accuracy and Coherence: Prospects for an Alethic Epistemology of Partial Belief". en. In: *Degrees of Belief*. Ed. by Franz Huber and Christoph Schmidt-Petri. Dordrecht: Springer Netherlands, 2009, pp. 263–297. DOI: 10.1007/978-1-4020-9198-8_11.

[17] H. Greaves. "Epistemic Decision Theory". en. In: *Mind* 122.488 (Oct. 2013), pp. 915–952. DOI: 10.1093/mind/fzt090.

[18] Richard Pettigrew. *Accuracy and the laws of credence*. First edition. OCLC: ocn923850407. Oxford, United Kingdom : New York, NY: Oxford University Press, 2016.

[19] Murray Shanahan. "Talking about Large Language Models". In: *Communications of the ACM* 67.2 (Feb. 2024), pp. 68–79. DOI: 10.1145/3624724.

[20] Qingxiu Dong et al. "A Survey on In-context Learning". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 1107–1128. DOI: 10.18653/v1/2024.emnlp-main.64.

[21] Fabian Falck, Ziyu Wang, and Chris Holmes. "Is In-Context Learning in Large Language Models Bayesian? A Martingale Perspective". In: ().

[22] Eitan Wagner and Omri Abend. *Express Your Doubts – Probabilistic World Modeling Should Not Be Based on Token Logprobs*. May 2026. DOI: 10.48550/arXiv.2505.02072.

[23] Alan Hájek. "The reference class problem is your problem too". en. In: *Synthese* 156.3 (June 2007), pp. 563–585. DOI: 10.1007/s11229-006-9138-5.

[24] Lily Hu. "Does calibration mean what they say it means; or, the reference class problem rises again". en. In: ().

[25] Juan C. Perdomo et al. *Performative Prediction*. en. arXiv:2002.06673 [cs]. Feb. 2021. DOI: 10.48550/arXiv.2002.06673.

[26] Juan C. Perdomo. *Revisiting the Predictability of Performative, Social Events*. arXiv:2503.11713 [cs]. July 2025. DOI: 10.48550/arXiv.2503.11713.

[27] Abhijeet Gupta et al. "Distributional Vectors Encode Referential Attributes". In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Lisbon, Portugal: Association for Computational Linguistics, 2015, pp. 12–21. DOI: 10.18653/v1/D15-1002.

[28] Jacqueline Harding. "Operationalising Representation in Natural Language Processing". In: *The British Journal for the Philosophy of Science* (Nov. 2023), p. 728685. DOI: 10.1086/728685.

[29] Guillaume Alain and Yoshua Bengio. *Understanding Intermediate Layers Using Linear Classifier Probes*. Nov. 2018. DOI: 10.48550/arXiv.1610.01644.

[30] Samantha Dies et al. *Epistemic Familiarity Is Associated With Belief Stability in Large Language Models*. July 2026. DOI: 10.48550/arXiv.2511.19166.

[31] Murray Shanahan. *Simulacra as Conscious Exotica*. July 2024. DOI: 10.48550/arXiv.2402.12422.

[32] janus. *Simulators*. Sept. 2022.